Communication between Applications

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Exercises

Setup

Before you begin, let's set up your system with all the necessary tools for these exercises.

System Tools and Python

First, update your package lists and install the core utilities: `curl` and `wget` for testing web services, and Python's package manager (`pip`) and virtual environment module (venv).

```
# 1. Update your package lists
sudo apt update; sudo apt full-upgrade -y; \
sudo apt autoremove -y; sudo apt autoclean

# 2. Install general tools, flatpak, and Python essentials
sudo apt install -y udisks2 curl wget \
flatpak python3-pip python3-venv

# 3. Add the Flathub repository
flatpak --user remote-add --if-not-exists \
flathub https://flathub.org/repo/flathub.flatpakrepo
```

Python Best Practices

Here you fill will a set of best practices to develop Python3 projects. For each Python exercise, please follow these steps:

1. Create a new directory for the project (e.g., `mkdir ex01 && cd ex01`).
2. Create an isolated virtual environment:

   ```
   python3 -m venv venv
   ```
3. Activate the environment:

   ```
   source venv/bin/activate
   ```
4. Create a `requirements.txt` file (as specified in each exercise) and install from it:

   ```
   pip install -r requirements.txt
   ```
5. **Use the `logging` module** instead of `print()` for all your status messages.

   ```
   import logging
   logging.basicConfig(level=logging.INFO, format='%(message)s')
   logger = logging.GoogletLogger(__name__)

   logger.info("This is an info message.")
   ```

Network warning

You will typically use Eduroam to access the internet during classes. For most activities, this is sufficient; however, this network (managed by the university) blocks communication between student **equipment**. Keep this in mind, without being connected into another network communication between pairs is limited.

Exercise 1: UDP File Transfer

Goal: Explore the provided `file_transfer.py` script. Understand how it uses `asyncio` to create a persistent server that can handle multiple file uploads from clients.

Details:

- **Server:** The server is persistent. It uses a `dict` to manage file transfers from different clients, keyed by their IP and port (`addr`).
- **Client:** The client sends the file metadata (filename, size) first, then sends the data chunks, showing a progress bar using `tqdm`.
- **Protocol:** The script uses a simple newline-based protocol:
  - `START:<total_chunks>:<total_size>:<filename>`
  - `DATA:<chunk_num>:<data_chunk>`
  - `END`
  - The server responds with `ACK_ALL` or `ACK_FAIL`.

Instructions:

1. Create a new directory `ex01` and move into it `cd ex01`.
2. Download the solution code into this directory.
3. Activate a venv and install the requirements:

   ```
   python3 -m venv venv
   source venv/bin/activate
   pip install -r requirements.txt
   ```

4. Create a file to send, e.g., `echo "This is a UDP test file." > test.txt`.
5. **Run the Server (Terminal 1):**

   ```
   python file_transfer.py receive --port 9999
   ```

6. **Run the Client (Terminal 2):**

   ```
   python file_transfer.py send test.txt --host 127.0.0.1 --port 9999
   ```

Exercise 2: Remote Tic-Tac-Toe

Goal: Analyze the provided `main.py` script to see how `asyncio` can be integrated with a GUI library like Pygame to create a networked application.

Details:

- **GUI Menus:** The script uses Pygame to draw all its own menus. It does not use `argparse`.
- **Async Game Loop:** The main `while running:` loop is `async`. It yields control to the `asyncio` event loop by calling `await asyncio.sleep(1/FPS)`.
- **Networking:** The script uses `asyncio.start_server` (for the host) and `asyncio.open_connection` (for the client) to create reliable TCP streams.
- **Error Handling:** The `run()` and `close_connection()` functions use `try...finally` and handle `asyncio.CancelledError` to ensure the application shuts down cleanly.

Instructions:

1. Create a new directory `ex02` and move into it `cd ex02`.
2. Download the solution code into this directory.
3. Activate a venv and install the requirements:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

4. **Run the Host (Player X):**

   ```
   python main.py
   ```

   - In the GUI, click "Host Game" -> enter a port (e.g., 8888) -> Press Enter.

5. **Run the Client (Player O):**

   ```
   python main.py
   ```

   - In the GUI, click "Join Game" -> enter the host's IP (`127.0.0.1` if on the same machine) -> Press Enter -> enter the port (8888) -> Press Enter.

Exercise 3: FastAPI Caching Service

Goal: Run and test the provided `main.py` script to understand how to build a high-performance, caching API endpoint.

Details:

- **Endpoint:** The script provides a `GET /ip/{ip_address}` endpoint.
- **Cache:** It uses a local `ip_cache.json` file.
- **Logic:** It checks the `timestamp` of a cached entry against a `CACHE_DEADLINE_SECONDS`.
- **External API:** If the cache is stale or missing, it uses the `requests` library to fetch live data.

Instructions:

1. Create a new directory `ex03` and move into it `cd ex03`.
2. Download the solution code into this directory.
3. Activate a venv and install the requirements:

   ```
   python3 -m venv venv
   source venv/bin/activate
   pip install -r requirements.txt
   ```

4. **Run the Server:**

   ```
   uvicorn main:app --reload
   ```

5. **Test the Service (in a new terminal):**

   - **Test 1 (Cache Miss):**

     ```
     # Private IP (has to fail)
     curl http://127.0.0.1:8000/ip/192.168.132.132

     # Google DNS
     curl http://127.0.0.1:8000/ip/8.8.8.8

     # Public IP from MEO
     curl http://127.0.0.1:8000/ip/144.64.3.83

     # UA
     curl http://127.0.0.1:8000/ip/193.137.169.135

     # Static IP from São Tomé
     curl http://127.0.0.1:8000/ip/197.159.166.30
     ```

     (Check the server logs; it should say "Querying external API".)

   - **Test 2 (Cache Hit):**

     ```
     # Private IP (has to fail)
     curl http://127.0.0.1:8000/ip/192.168.132.132
     ```

```
# Google DNS
curl http://127.0.0.1:8000/ip/8.8.8.8

# Public IP from MEO
curl http://127.0.0.1:8000/ip/144.64.3.83

# UA
curl http://127.0.0.1:8000/ip/193.137.169.135

# Static IP from São Tomé
curl http://127.0.0.1:8000/ip/197.159.166.30
```

(Check the server logs; it should say "Returning cached data".)

Exercise 4: Pub/Sub Chat

Goal: Use Docker to run an MQTT broker and connect to it with a pure JavaScript client to create a "serverless" chat application.

Details:

- **No Python Server:** You will not write *any* server code. The Mosquitto broker *is* the server.
- **Broker:** The `docker-compose.yml` file starts Mosquitto and loads `mosquitto.conf`.
- **Configuration:** The `.conf` file enables anonymous access and opens port `9001` for **MQTT-over-WebSockets**.
- **Client:** The `chat_client.html` file uses the **MQTT.js** library (loaded from a CDN) to connect to `ws://localhost:9001`. It implements a Pub/Sub chat.

Instructions:

1. Create a new directory `ex04` and move into it `cd ex04`.
2. Download the code into the same directory.
3. **Start the Broker:**

   ```
   docker compose up -d
   ```

4. **Test the Client:**
   - Open `http://localhost:8080/` in your web browser.
   - Open `http://localhost:8080/` in a *second* browser tab or window.
   - Enter different usernames and connect. Messages sent in one window should appear in the other.
   - You can use the `TheOffice` network to let chat with other students.

Bonus Exercise: The Classic Echo Server

Goal: Write a simple Echo Server in Python using the built-in `socket` module. This is the "Hello, World!" of network programming.

Task: This is the only exercise where you **must write the code yourself.**

Create a single Python script `echo_server.py`. The script should be able to run in one of two modes using `argparse`:

1. `python echo_server.py tcp --port <num>`
2. `python echo_server.py udp --port <num>`

Requirements:

- **TCP Mode:** The server must listen on the given port, accept a client connection, and `recv` data from the client. It must then `sendall` the *exact same data* back. It must handle clients disconnecting gracefully.
- **UDP Mode:** The server must bind to the given port, `recvfrom` a datagram, and `sendto` the *exact same data* back to the address it came from.
- You must write this code from scratch. **Do not use `asyncio` for this exercise.**
- Test your TCP server with `netcat`: `nc 127.0.0.1 <port>`.

- Test your UDP server with `netcat`: `nc -u 127.0.0.1 <port>`.

Helpful Documentation:

- **Python `socket` Module:** https://docs.python.org/3/library/socket.html
- **Python Socket Programming HOWTO Guide:** https://docs.python.org/3/howto/sockets.html